

EE-170L Computer Programming Lab

Laboratory Report — No. 12

Pointers in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	12
Topic	Pointer Declaration, Referencing & Dereferencing Operators
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand how computer memory is organized as addressable bytes.
- Recognize that every variable occupies a specific block of memory with a unique address.
- Declare pointer variables using the * operator and understand their role.
- Use the referencing operator & to obtain the memory address of a variable.
- Use the dereferencing operator * to access or modify the value at a pointed-to address.
- Apply pointer arithmetic to traverse array elements using pointer variables.

2. Theoretical Background

2.1 Computer Memory

Computer memory is made up of bytes. Each byte has a unique number called its address. When a variable is declared, the compiler reserves a block of consecutive bytes to store its value. The size of the block depends on the data type:

- int — typically 4 bytes on a 32-bit system.
- float — typically 4 bytes.
- char — 1 byte.
- double — typically 8 bytes.

When a variable is used in code, two steps happen automatically: (1) the compiler locates the variable's address, then (2) retrieves the value stored at that address.

2.2 What is a Pointer?

A pointer is a variable that stores the memory address of another variable, rather than a value in the traditional sense. Because a pointer holds an address, it has two parts:

- **The pointer itself:** holds the address of another variable.
- **The pointed-to location:** holds the actual data value at that address.

Pointers are typed — an int pointer can only point to an int variable, a float pointer to a float, and so on.

2.3 Pointer Declaration

A pointer variable is declared by placing an asterisk (*) before the variable name:

Syntax:

```
dataType *variableName;

// Examples:
int *ptr;    // pointer to an integer
float *fp;   // pointer to a float
char *cp;    // pointer to a character
```

- int *ptr declares ptr as a pointer variable that will store the address of an integer.
- The * in declaration means 'this is a pointer', not a multiplication or dereference.

2.4 Referencing Operator (&)

The & operator retrieves the memory address of a variable. It is used to assign an address into a pointer:

```
int k    = 5;        // ordinary integer variable
int *ptr;            // pointer variable — holds an address
ptr = &k;            // & retrieves the address of k; now ptr 'points to' k
```

- &k does not read the value 5; it reads the address where 5 is stored (e.g., 0x0062).
- After ptr = &k, ptr contains that address and is said to point to k.

2.5 Dereferencing Operator (*)

When * is used on an already-declared pointer (not in a declaration), it accesses the value at the address the pointer holds. This is called dereferencing:

```
int k    = 5;
int *ptr = &k;    // ptr holds the address of k

*ptr = 7;          // dereference: write 7 to the address ptr holds → k becomes 7
cout << *ptr;      // dereference: read the value at ptr's address → prints 7
```

- *ptr on the left of = writes a value to the pointed-to address.
- *ptr on the right of = or in an expression reads the value at the pointed-to address.

3. Example Programs

3.1 Reading and Writing via Pointers

```
#include <iostream>
using namespace std;
int main()
{
    int a = 5;                // declare integer variable a
    int b = 15;               // declare integer variable b
    int *p1, *p2;             // declare two integer pointer variables

    p1 = &a;                  // p1 now holds the address of a
    p2 = &b;                  // p2 now holds the address of b

    *p1 = 10;                 // dereference p1: set a = 10 (was 5)
    *p2 = *p1;                // dereference both: set b = value of a = 10
```

```

    *p1 = 20;                                // dereference p1: set a = 20

    cout << "a = " << a << endl;           // print a: expected 20
    cout << "b = " << b << endl;           // print b: expected 10

    return 0;
}

```

Output:

```

a = 20
b = 10

```

Trace:

- After `*p1 = 10`: `a = 10`, `b = 15`.
- After `*p2 = *p1`: `b = 10` (copies the current value of `a` into `b`).
- After `*p1 = 20`: `a = 20`, `b` remains 10.

3.2 Displaying Address and Value Using a Pointer

```

#include <iostream>
using namespace std;
int main()
{
    int num = 42;                                // integer variable with value 42
    int *ptr = &num;                            // pointer initialized to address of num

    // Display the address stored inside the pointer
    cout << "Address of num (via &num) : " << &num << endl;
    cout << "Address stored in ptr      : " << ptr << endl;

    // Display the value at the address the pointer holds
    cout << "Value of num              : " << num << endl;
    cout << "Value via *ptr            : " << *ptr << endl;

    return 0;
}

```

Output (address will vary by run):

```

Address of num (via &num) : 0x61ff0c
Address stored in ptr    : 0x61ff0c
Value of num            : 42
Value via *ptr          : 42

```

4. Lab Tasks & Solutions**Task 1 — Pointer Declaration and Initialization**

Declare an integer variable 'num' initialized with a value of your choice. Declare a pointer 'ptr' of type integer pointing to num. Display the value of num directly and via the dereference operator.

```

#include <iostream>
using namespace std;
int main()
{
    // Step 1: Declare and initialize an integer variable
    int num = 75;                                // num holds the value 75

    // Step 2: Declare an integer pointer variable
    int *ptr;                                    // ptr will store an address of an int
}

```

```

// Step 3: Initialize the pointer to point to num
ptr = &num; // & retrieves the address of num

// Step 4: Display address information
cout << "Address of num      : " << &num << endl; // memory address of num
cout << "Value stored in ptr: " << ptr << endl; // same address, stored in
ptr

// Step 5: Display values - directly and through the pointer
cout << "Value of num      : " << num << endl; // direct access
cout << "Value via *ptr    : " << *ptr << endl; // dereference - reads
value at address

// Step 6: Modify num through the pointer and verify
*ptr = 100; // dereference to change num through ptr
cout << "After *ptr = 100:" << endl;
cout << "num is now      : " << num << endl; // num should now be 100

return 0;
}

```

Output (address will vary):

```

Address of num      : 0x61ff08
Value stored in ptr: 0x61ff08
Value of num       : 75
Value via *ptr     : 75
After *ptr = 100:
num is now        : 100

```

How it works:

- `ptr = &num` stores the address of `num` in the pointer variable `ptr`.
- `*ptr` reads (dereferences) the value at that address, returning 75.
- `*ptr = 100` writes 100 directly to `num`'s memory location — the original variable changes.

Task 2 — Pointer Arithmetic — Traversing an Array

Declare an integer array 'numbers' with values of your choice. Declare a pointer 'ptr' pointing to the first element. Use pointer arithmetic to access and display every element.

```

#include <iostream>
using namespace std;
int main()
{
    // Step 1: Declare and initialize the integer array
    int numbers[5] = {10, 20, 30, 40, 50}; // 5-element array

    // Step 2: Declare a pointer and point it to the first element
    int *ptr = numbers; // 'numbers' decays to address of numbers[0]

    // Step 3: Display each element using pointer arithmetic
    // ptr + i moves the pointer i positions forward (each step = sizeof(int)
    bytes)
    cout << "Array elements using pointer arithmetic:" << endl;
    for (int i = 0; i < 5; i++)
    {
        // *(ptr + i) dereferences the address (ptr + i) to get element at index
        i
        cout << "numbers[" << i << "] = " << *(ptr + i) << endl;
    }
}

```

```

// Step 4: Also show traversal using pointer increment
cout << "\nUsing ptr++ traversal:" << endl;
ptr = numbers;                // reset pointer back to first element
for (int i = 0; i < 5; i++)
{
    cout << *ptr << " ";      // dereference current position
    ptr++;                    // advance pointer to next int (adds
sizeof(int) bytes)
}
cout << endl;

return 0;
}

```

Output:

```

Array elements using pointer arithmetic:
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50

Using ptr++ traversal:
10 20 30 40 50

```

How it works:

- An array name decays to a pointer to its first element — `ptr = numbers` is equivalent to `ptr = &numbers[0]`.
- `*(ptr + i)` computes the address `ptr + i*sizeof(int)` and dereferences it — same as `numbers[i]`.
- `ptr++` increments the pointer by `sizeof(int)` bytes (typically 4), moving to the next element.

5. Pointer Operators — Quick Reference

Operator	Name	Used In	Effect
*	Asterisk in declaration	<code>int *ptr;</code>	Declares <code>ptr</code> as a pointer variable
&	Referencing (Address-of)	<code>ptr = &x;</code>	Gets the memory address of variable <code>x</code>
*	Dereference	<code>*ptr = 5;</code> or <code>cout << *ptr;</code>	Reads or writes the value at <code>ptr</code> 's address
<code>ptr++</code>	Pointer increment	<code>ptr++;</code>	Moves pointer forward by <code>sizeof(type)</code> bytes
<code>ptr+i</code>	Pointer offset	<code>*(ptr+i)</code>	Accesses the element <code>i</code> positions ahead

6. Conclusion

This lab introduced pointers — one of the most powerful and fundamental features of C++. Key takeaways:

- **Memory Model:** Every variable occupies one or more bytes at a specific address. Pointers expose this addressing mechanism directly.
- **Pointer Declaration:** `dataType *ptr;` declares a pointer. The type tells the compiler how many bytes to read when the pointer is dereferenced.
- **& Operator:** Retrieves the address of a variable. Used to initialize or reassign a pointer.
- *** Operator:** Dereferences a pointer to read or write the value at the address it holds. Can also modify the original variable indirectly.
- **Pointer Arithmetic:** Adding an integer to a pointer moves it by $n * \text{sizeof}(\text{type})$ bytes, making array traversal possible without index notation.

Pointers are the foundation of dynamic memory allocation, linked data structures, and efficient function parameter passing — all critical topics in advanced C++ programming.